CREAR NOVA APLICACIÓ

Professor: Daniel Jesús García Martínez

Versió: gener 2025

iEbre INSTITUT DE L'EBRE TORTOSA	Departament d'Informàtica	DAM MP10

CREAR NOVA APLICACIÓ EN ODOO **3**
INICIALITZAR __MANIFEST__.PY 3
INICIALITZAR __INIT__.PY 5
CREAR NOU MODEL **6**
AFEGIR ATRIBUTS AL MODEL **8**
ACTIVITAT 10
SEGURETAT EN MÒDULS **11**
AFEGIR UI **13**
ACCIONS 13
MENÚS 14
DEFINIR PROPIETATS DE CAMPS 17
ACTIVITAT 18

CREAR NOVA APLICACIÓ EN ODOO

Per crear una nova aplicació seguirem el tutorial:

https://www.odoo.com/documentation/master/developer/tutorials/server_framework_101/02_newapp.html

Dins del repositori “odoo-tutorials”, creem un directori “estate”:

A l'interior creem dos fitxers:

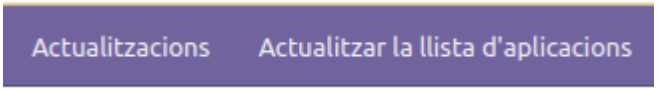
```
__manifest__.py
__init__.py
```

INICIALITZAR __MANIFEST__.PY

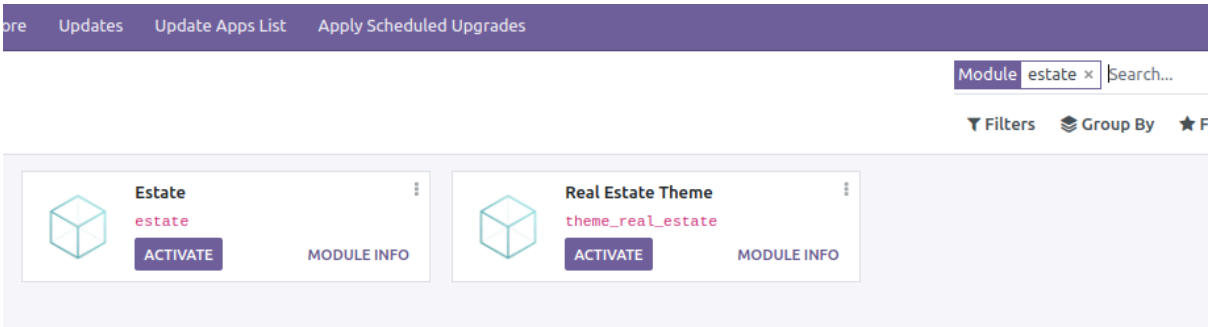
En primer lloc, cal inicialitzar el fitxer “__manifest__.py” amb la informació bàsica d’una aplicació:

```
{
   "name": "Estate",  # The name that will appear in the App list
   "depends": ["base"],  # dependencies
}
```

Per comprovar si la nova aplicació apareix, hem d’actualitzar la llista d’aplicacions (haurem d’estar en mode desenvolupador):



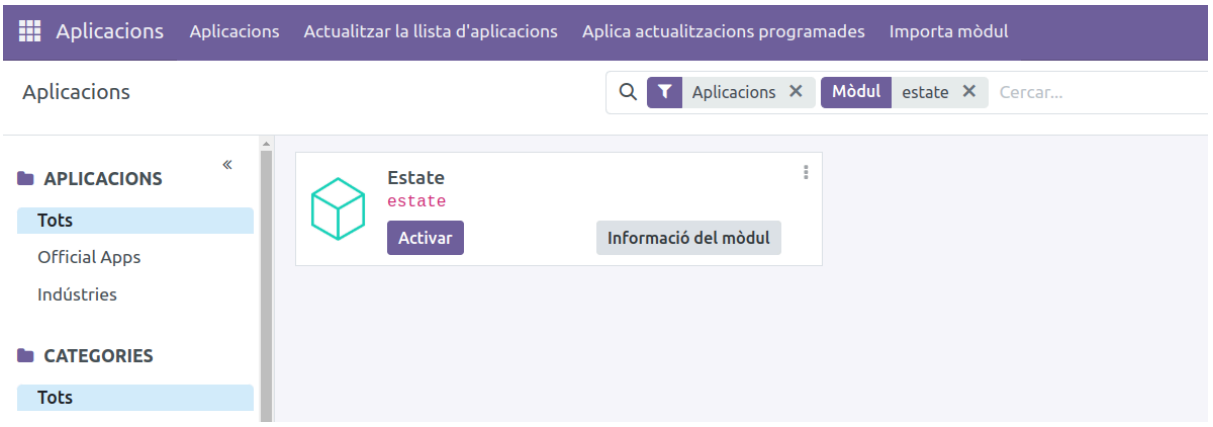
Si no apareix l'aplicació, hem de treure el filtre “Apps” i cercar pel nom:



Per corregir-ho i que es reconegui com aplicació, afegim la clau “application”:

```
{
   "name": "Estate",  # The name that will appear in the App list
   "application": True,  # This line says the module is an App, and not a module
   "depends": ["base"],  # dependencies
}
```

Seguidament, actualitzem aplicacions i comprovem que ja apareix dins del filtre “Apps”:



Una configuració completa de “__manifest__.py” seria:

```
{
   "name": "Estate",  # The name that will appear in the App list
   "version": "1.0",  # Version
   "application": True,  # This line says the module is an App, and not a module
   "depends": ["base"],  # dependencies
   "data": [

   ],
```

```
    "installable": True,
    'license': 'LGPL-3',
}
```

INICIALITZAR __INIT__.PY

El fitxer __init__.py ha de definir el paquet “estate” important el paquet “models”:

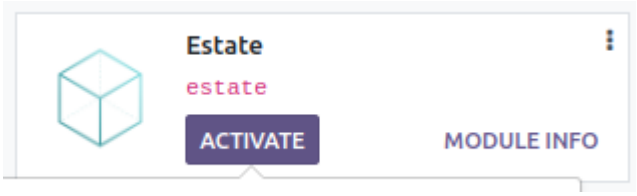
```
from . import models
```

CREAR NOU MODEL

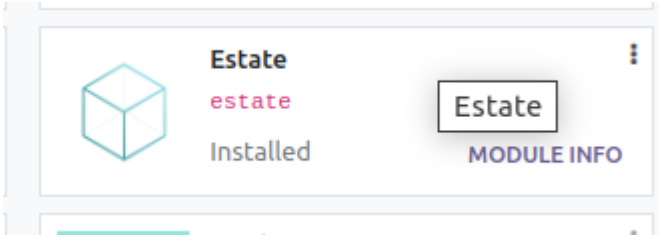
Seguirem el capítol del tutorial:

https://www.odoo.com/documentation/18.0/developer/tutorials/server_framework_101/03_basicmodel.html

De moment només disposem d'una aplicació creada, però està buida. En tot cas, ja la podem activar:

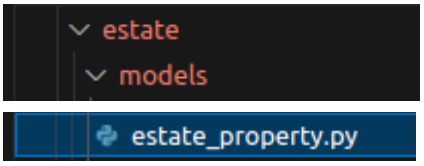


No disposem de cap menú per accedir, però està instal·lada:



Crearem un model que permetrà crear una primera taula en base de dades anomenada "estate_property". El mecanisme utilitzat per mapejar el model i la taula és l'ORM (Object Relational Mapping) d'Odoo (és un ORM propi).

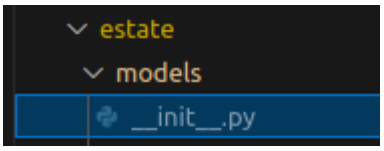
Creem un fitxer "estate_property.py" dins de la carpeta "models":



Dins de "estate_property.py" definim una classe que ha d'estendre de "Model". Inicialitzem un atribut "_name" que identificarà el nom de la taula a crear en base de dades:

```
from odoo import fields, models
class EstateProperty(models.Model):
    _name = 'estate.property'
```

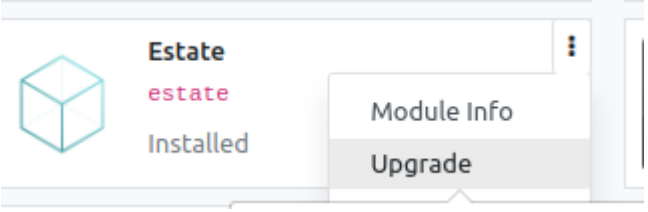
Per incloure el nou mòdul en el paquet, cal crear "__init__.py" dins de "models":



I cal inicialitzar-lo amb el mòdul Python creat.

```
from . import estate_property
```

Com hi ha canvis en el model, cal fer un "upgrade" o actualització del mòdul:



Els canvis en el model s'actualitzen. Podríem comprovar els canvis en la base de dades.

AFEGIR ATRIBUTS AL MODEL

A continuació afegirem diferents camps al model ja creat. Cada atribut tindrà una correspondència entre l'atribut de Python i el camp de la taula en base de dades.

Es poden crear atributs de dues grans categories: simples i relacionals.

Alguns dels simples són:

- Boolean
- Float
- Integer
- Char
- Text
- Date
- Selection

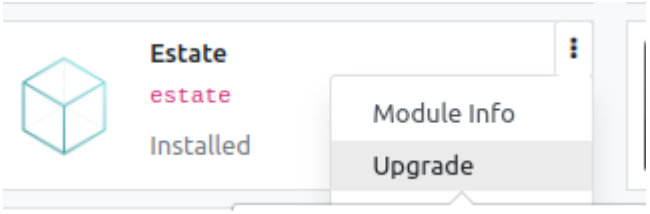
Els relacionals, per establir relacions amb altres models, serien:

- Many2one
- Many2many
- One2many

Afegim nous atributs a EstateProperty:

```
class EstateProperty(models.Model):
    _name = 'estate.property'
    _description = 'Model per estate_property'
    name = fields.Char('Propietat Immobiliària', required=True)
```

Cal actualitzar l'aplicació:



Si consultem la taula a la base de dades, la trobem creada amb els atributs:

estate_property

General | Columns | Advanced | Constraints | Parameters | Security | SQL

Columns

Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id	integer			✓	✓	nextval('estate
create_uid	integer					
write_uid	integer					
name	character varying			✓		
create_date	timestamp without time zone					
write_date	timestamp without time zone					

En la definició d'un atribut, el primer paràmetre és l'etiqueta que apareixerà en la UI.

```
name = fields.Char('Propietat Immobiliària', required=True)
```

També es pot identificar un atribut com obligatori. Això es tradueix en base de dades en una columna definida com a NOT NULL.

Odoo també crea algunes columnes en base de dades de manera automàtica, com id, create_uid o write_uid. La columna "id" és la definida com a clau primària en base de dades (PK).

id	integer			✓	✓	nextval('estate
create_uid	integer					
write_uid	integer					

També es creen camps per la data de creació i modificació:

create_date	timestamp withou					
write_date	timestamp withou					

ACTIVITAT

Afegir nous atributs definint el tipus de dades més adient, l'etiqueta, si són obligatoris o no i finalment comprovar l'actualització de la taula en base de dades:

description

date_availability

selling_price (obligatori)

postalcode

bedrooms

SEGURETAT EN MÒDULS

Les dades d'un mòdul es poden carregar mitjançant fitxers CSV. En general, aquests fitxers es guarden en un directori "data" dins del directori del mòdul. En el cas anterior, en "estate/data".

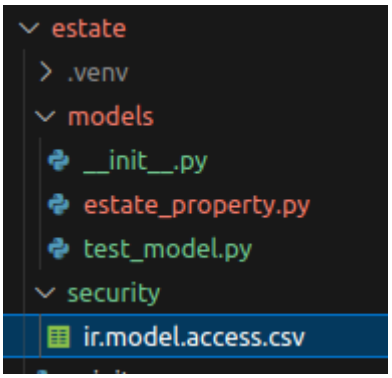
Aquests fitxers de dades s'han de declarar en el fitxer "__manifest__.py" del mòdul Odoo.

Un tipus específic de dades són les que configuren la seguretat. Aquestes se situen en un directori "estate/security".

Si no s'han definit drets d'accés, cap usuari podrà accedir a les dades. Això es mostra en els logs:

```
2023-12-26 21:11:33,006 1 INFO empresa_test odoo.modules.registry: module estate: creating or updating database tables
2023-12-26 21:11:33,161 1 WARNING empresa_test odoo.modules.loading: The models ['test_model', 'estate_property'] have no access rules in module estate, consider adding some,
 like:
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
estate.access_test_model,access_test_model,estate.model_test_model,base.group_user,1,0,0,0
estate.access_estate_property,access_estate_property,estate.model_estate_property,base.group_user,1,0,0,0
```

Afegim la configuració d'accés per estate_property. Primer creem un fitxer "ir.model.access.csv" dins d'un directori "security":



I afegim el contingut al fitxer:

```
id,name,model_id/id,group_id/id,perm_read,perm_write,perm_create,perm_unlink
access_estate_property,access_estate_property,model_estate_property,base.group_user,1,1,
1,1
```

Per entendre els diferents camps:

- id és un identificador extern.
- name és el nom d'aquest ir.model.access.
- model_id/id fa referència al model al qual apliquen els drets. La forma estàndard és model_<model_name>, on <model_name> és l'atribut _name del model reemplaçant "." per "_".
- group_id/id fa referència al grup de usuaris amb accés.
- perm_read,perm_write,perm_create,perm_unlink: permisos de lectura, escriptura, creació i esborrat.

Per últim, afegim el fitxer a "__manifest__.py"

```
{
   "name": "Estate",  # The name that will appear in the App list
   "application": True,  # This line says the module is an App, and not a module
   "depends": ["base"],  # dependencies
   'data': [
       'security/ir.model.access.csv',
   ]
}
```

Reiniciem l'aplicació per aplicar els canvis. Podem comprovar si els logs encara mostren el missatge anterior.

AFEGIR UI

De moment l'aplicació Estate està instal·lada, però no hi ha cap manera d'accedir-hi per la interfície de l'aplicació.

A continuació definirem el necessari per poder accedir a les vistes de l'aplicació.

Per definir vistes, farem servir fitxers "xml" i els afegirem al manifest de manera similar als "csv" comentats anteriorment.

En el cas de les vistes, els fitxers es guardaran en el directori "views" dins del mòdul d'Odoo.

L'UI en Odoo es compon de tres elements bàsics: accions, menús i vistes.

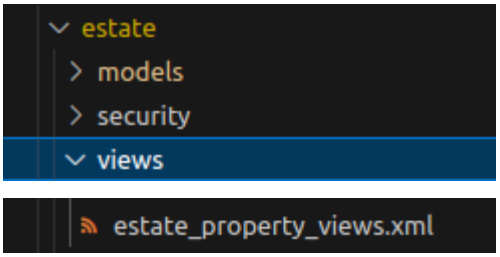
ACCIONS

Accions que poden llançar-se des d'elements de menú, botons en una vista o accions contextuals a un objecte.

La definició bàsica d'una acció és:

```
<record id="test_model_action" model="ir.actions.act_window">
    <field name="name">Test action</field>
    <field name="res_model">test_model</field>
    <field name="view_mode">list,form</field>
</record>
```

Creem una primera acció per "estate_property" i la ubiquem en el fitxer corresponent "estate_property_views.xml" dins del directori "views"



```xml
<?xml version="1.0"?>
<odoo>
   <record id="estate_property_action" model="ir.actions.act_window">
       <field name="name">Propietats</field>
       <field name="res_model">estate.property</field>
       <field name="view_mode">list,form</field>
   </record>
</odoo>
```

Afegim també la referència al manifest.

```python
{
   "name": "Estate",  # The name that will appear in the App list
   "application": True,  # This line says the module is an App, and not a module
   "depends": ["base"],  # dependencies
   'data': [
       'security/ir.model.access.csv',
       'views/estate_property_views.xml',
   ]
}
```

Actualitzem el mòdul. En els logs apareix carregat el fitxer:

```
INFO empresa_test odoo.modules.loading: loading estate/views/estate_property_views.xml
```

MENÚS

Per afegir un menú, farem servir la següent definició:

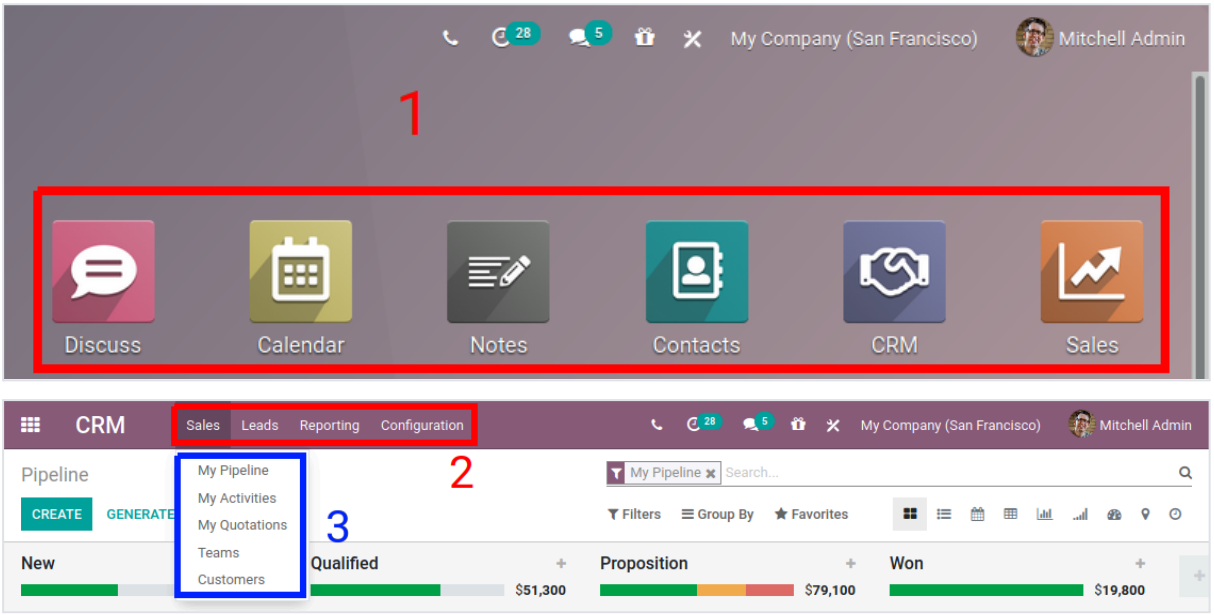
```xml
<menuitem id="test_model_menu_action" action="test_model_action"/>
```

El menú definit enllaça amb una acció prèviament definida.

Hi ha 3 nivells de menú:

1. Menú arrel, que es mostra en la selecció d'aplicacions.
2. Primer nivell de menú, que es mostra en la barra superior.

3. El menú d'accions que es mostra com desplegable del primer nivell.



Una estructura bàsica amb aquests nivells seria:

```
<menuitem id="test_menu_root" name="Test">
    <menuitem id="test_first_level_menu" name="First Level">
        <menuitem id="test_model_menu_action" action="test_model_action"/>
    </menuitem>
</menuitem>
```

Creem els menús per l'aplicació "Estate" creant un fitxer anomenat "estate_menus.xml" dins també de "views". L'inicialitzem amb:

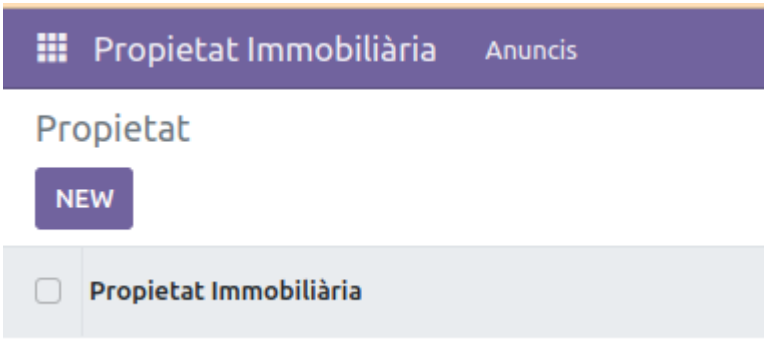
```
<?xml version="1.0"?>
<odoo>
    <menuitem id="estate_menu_root" name="Propietat Immobiliària">
        <menuitem id="estate_first_level_menu" name="Anuncis">
            <menuitem id="estate_menu_action" action="estate_property_action"/>
        </menuitem>
    </menuitem>
</odoo>
```

També actualitzem el manifest.

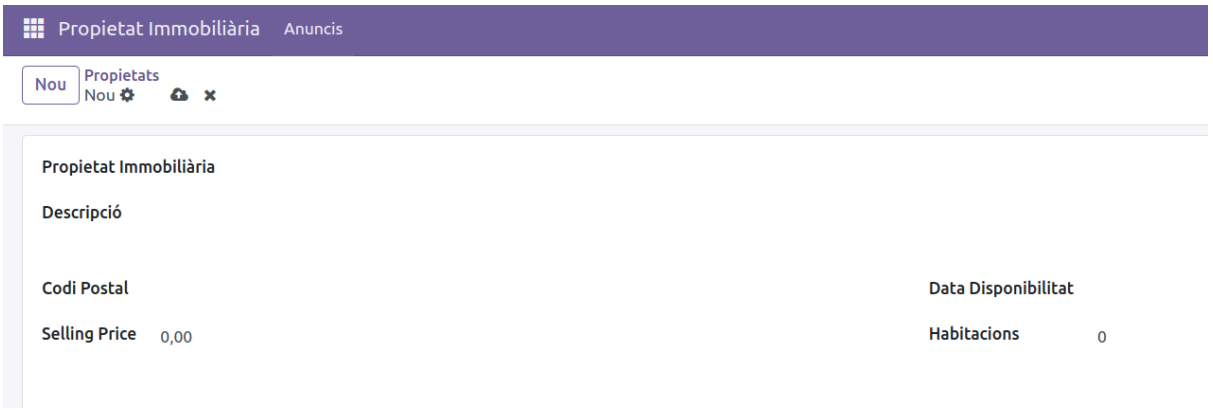
```
{
    "name": "Estate",  # The name that will appear in the App list
```

```
    "application": True,  # This line says the module is an App, and not a module
    "depends": ["base"],  # dependencies
    'data': [
        'security/ir.model.access.csv',
        'views/estate_property_views.xml',
        'views/estate_menus.xml',
    ]
}
```

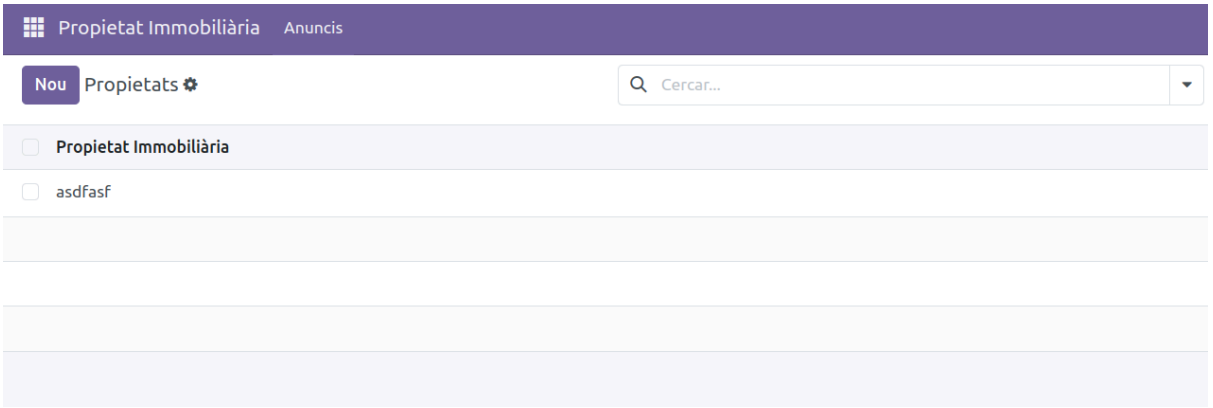
Com a resultat obtenim l'accés a una primera vista amb els menús definits:



Fins i tot podríem intentar crear el primer anunci de propietat:



Inserim dades i tornem al llistat. Podem veure en la llista el nou anunci creat:



DEFINIR PROPIETATS DE CAMPS

Podem indicar que un camp només és de lectura (readonly=True) i que no es copia quan es duplica un registre (copy=False).

Ho fem amb “bedrooms” i “date_availability”:

```
    date_availability = fields.Date('Data Disponibilitat',copy=False)
    bedrooms = fields.Integer('Habitacions',readonly=True)
```

Comprovem i veiem que “Habitacions” està en gris no permetent modificacions:

Si fem un duplicat, no es copien els valors de la data de disponibilitat.

Abans d’afegir valors per defecte a camps de tipus “Date”, podem instal·lar el paquet “python-dateutil”, útil per treballar amb dates. D’aquí podem obtenir la classe “relativedelta”. Abans d’utilitzar-lo l’hem d’instal·lar dins de l’entorn “venv” Python del workspace (crear-lo si no existeix):

```
  pip install python-dateutil
```

Ara podem definir valors per defecte, per exemple una data de disponibilitat de 3 mesos a partir de la data actual:

```
    date_availability = fields.Date(default=fields.Date.today()+relativedelta(months=3),
```

```
copy=False)
```

També podem definir camps predefinits com “active” o “state”. “active” identifica si un registre està actiu. Si es configura com no actiu, no apareixerà en el llistat.

ACTIVITAT

- Definiu valors per defecte per cadascun dels camps ja disponibles
- Definiu diferents valors predefinits per un nou camp “state” que permetrà seleccionar l’estat de l’anunci.
- Definiu un nou camp “active” de tipus booleà i comproveu l’efecte dels valors “False” i “True” en el llistat. Busqueu com continuar veient els registres independentment del valor d’aquest atribut. Aconseguiu mostrar els registres que existien abans d’afegir el nou camp.

Fins aquí una primera aplicació amb un model i les vistes bàsiques per interaccionar. En el següent capítol profunditzarem en com crear vistes personalitzades.